

# Teach Module 11: GitHub Copilot for Testing and Refactoring

This README uses the course document only to identify the Module 11 topic list. The teaching content below is written independently and does not reuse the course material.

## Module Focus

Module 11 covers practical use of GitHub Copilot or similar AI coding assistants for:

- JUnit test generation
- Mockito mock generation
- Refactoring suggestions and code smell detection
- Coverage review and gap identification
- Acceptance guidelines for AI-generated code
- AI-assisted test generation lab work

## Core Idea

Copilot can speed up test writing and refactoring, but it does not replace engineering judgment. Treat AI-generated code as a first draft. You are responsible for checking correctness, readability, edge cases, and maintainability.

## 1. JUnit Test Generation

JUnit is the standard testing framework for Java. A unit test checks one small piece of behavior.

Example class:

```
public class DiscountService {
    public double applyDiscount(double price, double percent) {
        if (price < 0) {
            throw new IllegalArgumentException("Price cannot be negative");
        }
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("Invalid discount");
        }
        return price - (price * percent / 100);
    }
}
```

Example JUnit tests:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class DiscountServiceTest {

    private final DiscountService service = new DiscountService();
```

```

@Test
void appliesDiscountCorrectly() {
    double result = service.applyDiscount(100.0, 20.0);

    assertEquals(80.0, result);
}

@Test
void throwsExceptionForNegativePrice() {
    assertThrows(IllegalArgumentException.class, () ->
        service.applyDiscount(-50.0, 10.0)
    );
}

@Test
void throwsExceptionForInvalidDiscountPercent() {
    assertThrows(IllegalArgumentException.class, () ->
        service.applyDiscount(100.0, 150.0)
    );
}
}

```

Useful Copilot prompt:

```

Generate JUnit 5 tests for DiscountService.
Include normal cases, boundary cases, and exception cases.

```

## 2. What Makes A Good Unit Test

A good test usually follows Arrange, Act, Assert:

```

// Arrange
DiscountService service = new DiscountService();

// Act
double result = service.applyDiscount(100.0, 20.0);

// Assert
assertEquals(80.0, result);

```

Strong tests check:

- Normal behavior
- Edge cases
- Invalid input
- Exceptions
- Boundary values
- Null handling when relevant

Weak tests only prove that code ran. For example:

```
assertNotNull(result);
```

This is usually less useful than:

```
assertEquals(80.0, result);
```

### 3. Mockito Mock Generation

Mockito is used when a class depends on another class, database, API, queue, email service, or payment service.

Example:

```
public class OrderService {
    private final PaymentClient paymentClient;

    public OrderService(PaymentClient paymentClient) {
        this.paymentClient = paymentClient;
    }

    public boolean placeOrder(double amount) {
        return paymentClient.charge(amount);
    }
}
```

Mockito test:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class OrderServiceTest {

    @Test
    void placesOrderWhenPaymentSucceeds() {
        PaymentClient paymentClient = mock(PaymentClient.class);
        when(paymentClient.charge(100.0)).thenReturn(true);

        OrderService orderService = new OrderService(paymentClient);

        boolean result = orderService.placeOrder(100.0);

        assertTrue(result);
        verify(paymentClient).charge(100.0);
    }
}
```

When reviewing AI-generated Mockito tests, ask:

- Is the correct dependency mocked?
- Is the test verifying behavior rather than implementation noise?
- Is the mock setup realistic?
- Would this test catch a real bug?

## 4. Refactoring With Copilot

Refactoring means improving code structure without changing behavior.

Code smell example:

```
public double calculate(double price, String customerType) {
    if (customerType.equals("VIP")) {
        return price * 0.8;
    } else if (customerType.equals("REGULAR")) {
        return price * 0.95;
    } else if (customerType.equals("GUEST")) {
        return price;
    }
    return price;
}
```

Problems:

- Raw strings are easy to mistype.
- The logic can grow messy.
- Invalid customer types are not handled clearly.

Cleaner version:

```
public enum CustomerType {
    VIP,
    REGULAR,
    GUEST
}
```

```
public double calculate(double price, CustomerType customerType) {
    return switch (customerType) {
        case VIP -> price * 0.8;
        case REGULAR -> price * 0.95;
        case GUEST -> price;
    };
}
```

Useful Copilot prompt:

```
Refactor this method to improve readability and type safety.
Do not change the behavior.
```

Important rule: refactor only when tests exist or when you can add tests first.

## 5. Coverage Review And Gap Identification

Code coverage tells you how much of your code was executed by tests. It does not prove test quality.

Weak coverage example:

```
service.applyDiscount(100, 20);
```

This executes the method but proves little unless you assert the expected result.

Better:

```
assertEquals(80.0, service.applyDiscount(100, 20));
```

When reviewing coverage, ask:

- Are all branches tested?
- Are exceptions tested?
- Are edge cases tested?
- Are important business rules tested?
- Are tests asserting expected outcomes?

## 6. Acceptance Guidelines For AI-Generated Code

Before accepting AI-generated tests or refactors, check:

- Does it compile?
- Does it test real behavior?
- Are assertions meaningful?
- Are edge cases included?
- Are names clear?
- Is the code simpler after refactoring?
- Did behavior stay the same?
- Are mocks used only where needed?
- Would another developer understand this test?

## Practice Exercises

### Exercise 1: Generate JUnit Tests

Create this class:

```
public class GradeCalculator {
    public String getGrade(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException("Invalid score");
        }
        if (score >= 90) return "A";
        if (score >= 80) return "B";
        if (score >= 70) return "C";
        if (score >= 60) return "D";
        return "F";
    }
}
```

```
}  
}
```

Practice:

- Ask Copilot to generate JUnit 5 tests.
- Add missing boundary tests manually.
- Test 89, 90, 100, 0, -1, and 101.

## Exercise 2: Mockito Mocking

Create:

```
public interface EmailService {  
    void sendEmail(String to, String subject, String body);  
}
```

```
public class UserRegistrationService {  
    private final EmailService emailService;  
  
    public UserRegistrationService(EmailService emailService) {  
        this.emailService = emailService;  
    }  
  
    public void register(String email) {  
        if (email == null || email.isBlank()) {  
            throw new IllegalArgumentException("Email is required");  
        }  
  
        emailService.sendEmail(  
            email,  
            "Welcome",  
            "Thanks for registering"  
        );  
    }  
}
```

Practice:

- Mock EmailService using Mockito.
- Verify that sendEmail() is called for a valid email.
- Verify that it is not called for invalid email input.

## Exercise 3: Refactor Messy Code

Start with:

```
public double calculateShipping(String country, double weight) {  
    if (country.equals("US")) {  
        if (weight <= 5) return 5.0;  
        else return 10.0;  
    } else if (country.equals("CA")) {
```

```
        if (weight <= 5) return 7.0;
        else return 15.0;
    } else {
        return 25.0;
    }
}
```

Practice:

- Write tests first.
- Ask Copilot to refactor the method.
- Check that all tests still pass.
- Improve null handling for `country` .

#### Exercise 4: Find Test Coverage Gaps

Use this method:

```
public boolean canWithdraw(double balance, double amount) {
    if (amount <= 0) {
        return false;
    }
    if (balance < amount) {
        return false;
    }
    return true;
}
```

Practice testing:

- Valid withdrawal
- Amount is zero
- Amount is negative
- Balance is less than amount
- Balance equals amount

Then ask Copilot:

What test cases are missing for this method?

Review whether Copilot's answer is useful.

#### Exercise 5: Accept Or Reject AI Code

Ask Copilot to generate tests for this:

```
public int divide(int a, int b) {
    return a / b;
}
```

Review:

- Did it test normal division?
- Did it test division by zero?
- Did it test negative numbers?
- Did it test uneven division like `5 / 2` ?
- Are the assertions meaningful?

## Exercise 6: Refactor With Safety Tests

Start with:

```
public boolean isStrongPassword(String password) {
    if (password == null) return false;
    if (password.length() < 8) return false;
    if (!password.matches("[A-Z].*")) return false;
    if (!password.matches("[0-9].*")) return false;
    return true;
}
```

Practice:

- Write tests before refactoring.
- Refactor for readability.
- Confirm behavior does not change.
- Add tests for null, short password, missing uppercase, missing digit, and valid password.

## Lab: AI-Assisted Test Generation And Refactoring

### Lab Goal

Use Copilot or any AI coding assistant to help generate tests, identify missing coverage, and refactor Java code safely.

### Scenario

You are working on a simple banking service. The service allows customers to deposit money, withdraw money, and transfer funds between accounts.

### Starter Code

Create `BankAccount.java` :

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double balance) {
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("Account number is required");
        }

        if (balance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative");
        }
    }
}
```

```

        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }

        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }

        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds");
        }

        balance -= amount;
    }
}

```

Create NotificationService.java :

```

public interface NotificationService {
    void sendNotification(String accountNumber, String message);
}

```

Create BankAccountService.java :

```

public class BankAccountService {
    private final NotificationService notificationService;

    public BankAccountService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void transfer(BankAccount from, BankAccount to, double amount) {
        if (from == null || to == null) {
            throw new IllegalArgumentException("Accounts are required");
        }
    }
}

```

```

    }

    from.withdraw(amount);
    to.deposit(amount);

    notificationService.sendNotification(
        from.getAccountNumber(),
        "Transfer of " + amount + " completed"
    );
}
}

```

## Part 1: Generate JUnit Tests

Use Copilot with:

```

Generate JUnit 5 tests for BankAccount.
Include deposit, withdraw, constructor validation, and exception cases.

```

Expected tests should cover:

- Creating an account with valid data
- Rejecting blank account number
- Rejecting negative initial balance
- Depositing valid amount
- Rejecting zero deposit
- Rejecting negative deposit
- Withdrawing valid amount
- Rejecting zero withdrawal
- Rejecting withdrawal larger than balance

## Part 2: Write Mockito Tests

Use Copilot with:

```

Generate Mockito tests for BankAccountService transfer method.
Verify notificationService.sendNotification is called after a successful transfer.

```

Expected tests should cover:

- Successful transfer reduces sender balance
- Successful transfer increases receiver balance
- Notification is sent after successful transfer
- Transfer fails if sender account is null
- Transfer fails if receiver account is null
- Transfer fails if amount exceeds sender balance
- Notification is not sent when transfer fails

Example test idea:

```

@Test
void transferSendsNotificationWhenSuccessful() {

```

```

NotificationService notificationService = mock(NotificationService.class);
BankAccountService service = new BankAccountService(notificationService);

BankAccount from = new BankAccount("A100", 500);
BankAccount to = new BankAccount("B200", 100);

service.transfer(from, to, 150);

verify(notificationService).sendNotification(
    eq("A100"),
    contains("Transfer of 150.0 completed")
);
}

```

### Part 3: Find Coverage Gaps

Ask Copilot:

Review these tests and identify missing test cases or weak assertions.

Look for gaps like:

- Transfer with zero amount
- Transfer with negative amount
- Notification should not be called on failure
- Account balances should remain unchanged when transfer fails
- Constructor should reject null account number

### Part 4: Refactor The Code

Ask Copilot:

Refactor this code to reduce duplication and improve readability without changing behavior.

Possible refactor:

```

private void validatePositiveAmount(double amount, String message) {
    if (amount <= 0) {
        throw new IllegalArgumentException(message);
    }
}

```

Then update:

```

public void deposit(double amount) {
    validatePositiveAmount(amount, "Deposit amount must be positive");
    balance += amount;
}

public void withdraw(double amount) {
    validatePositiveAmount(amount, "Withdrawal amount must be positive");
}

```

```
    if (amount > balance) {  
        throw new IllegalArgumentException("Insufficient funds");  
    }  
  
    balance -= amount;  
}
```

## Part 5: Acceptance Review

Before accepting AI-generated code, answer:

- Do all tests compile?
- Do all tests pass?
- Are assertions meaningful?
- Are exception cases tested?
- Are mocks verifying useful behavior?
- Did refactoring preserve behavior?
- Is the code easier to read?
- Did Copilot introduce unnecessary complexity?

## Lab Deliverables

Submit:

1. BankAccount.java
2. BankAccountService.java
3. NotificationService.java
4. BankAccountTest.java
5. BankAccountServiceTest.java
6. Short notes answering:
  - What did Copilot generate well?
  - What did you have to fix?
  - What test cases were missing?
  - What refactoring did you accept or reject?

## Success Criteria

You are done when:

- All JUnit tests pass.
- Mockito verifies notification behavior.
- Failed transfers do not send notifications.
- Refactored code behaves the same as before.
- You can explain why each AI-generated test is useful.